

LLMs in Research Workflows

From research task to repeatable workflow on AWS or HPC

AI/ML Bootcamp 2026

Ryan Watkins, PhD
Professor, Educational Technology (MA)
Lead, Human-Technology Collaboration (PhD)
Associate Director, GW Trustworthy AI
rwatkins@gwu.edu | www.ryanrwatkins.com

Task → Context → Harness → Scale → Skills

Why this session

- Many researchers are already using LLM chatbots informally.
- Fewer have a repeatable workflow they can trust and reuse.
- Reproducible research requires more systematic use and documentation.
- This content is very new and reflects rapidly changing AI capabilities.
- This session focuses on building usable research workflows, not just trying tools.
- We will utilize open-weights models that you can run on GW's Amazon Web Services (AWS) or High Performance Computing (HPC) infrastructure.

Workshop outcomes

You will be able to...

1. identify a useful LLM-enabled task in your research
2. align LLM use with trustworthy research workflows
3. create better context for that task
4. build a small workflow in Jupyter using Ollama
5. save outputs and basic metadata
6. think about scaling with sbatch

The throughline

Research task → Context → Harness → Scale → Skills

Task: What are you trying to get done?

Context: What does the model need in order to do the task well?

Harness: What is the smallest repeatable workflow around the model?

Scale: How do you run it repeatedly on AWS or HPC?

Skills: Which repeated parts can become reusable modules?

Start with the task, not the tool

- Examples:
 - extract structured information from abstracts
 - summarize notes or memos
 - compare outputs across conditions
 - generate draft text from structured inputs
 - flag cases for human review
 - suggest preliminary qualitative codes for short excerpts
 - turn rough lab/meeting notes into a structured memo, including project risks
 - classify stakeholder comments or survey responses
 - extract key information from weekly public health reports or updates
 - simulate hundreds of user experiences using synthetic personas

Pods

Pod norms:

- solve together
- use the chatbot as a helper, not a substitute for understanding
- advanced students support the pod, then do stretch tasks

Pod forming:

- Work with people sitting around you

Build good context

Trustworthy outputs depend on good context:

- task instructions
- examples
- output format
- schemas
- sample inputs
- minimal documentation
- available tools, functions, or reusable skills the workflow can call
- static or dynamic context selected for the task at hand
- cautions and guardrails

Minimum viable context pack

- task statement
- sample input
- ideal output example
- schema / structure
- caution or guardrail

Example minimum viable context pack

Task statement: Extract key study information from an abstract into a structured record.

Sample input: One article abstract.

Ideal output example: A JSON (or table row) with topic, population, method, and finding.

Schema / structure: topic, population, method, main_finding, limitations_or_missing_info

Guardrail: Do not infer unstated details. Use “not stated.” if details are not available.

Pod Activity: create a context pack

Activity

Recommended time: 25 minutes

- **Milestone 1: Map Your Own Workflow (Individual | 5 mins)**
 - *Action:* Quickly list 3–5 primary tasks in your own research. Identify *one* simple, repeating task where an LLM could help, and sketch the input/output.
- **Milestone 2: Pitch & Pick One (Pod | 7 mins)**
 - *Action:* Share your task with your pod (1 min per person). Together, select **one** concrete, bounded task for the pod to build out today.
- **Milestone 3: Build the Minimum Viable Context Pack (Pod | 13 mins)**
 - *Action:* Open the **Pod Worksheet**. For your chosen task, define the *Task Statement, Sample Input, Ideal Output, Schema, and Guardrails*.
 - Worksheet URL: <https://github.com/glenamac/BootcampSquared/>

From plan to workflow in Jupyter on AWS

- Instantiate your harness plan in a starter notebook.
- The notebook includes sections for
 - loading input
 - defining the prompt template
 - testing one example
 - running multiple examples
 - saving outputs
 - saving metadata

HPC in research: what it is and when to use it

- HPC is shared at GW research computing infrastructure.
- AWS is shared at Amazon computing infrastructure.
- Supports larger datasets, repeated runs, batch jobs, open/local models via Ollama, and workflows needing more CPU, GPU, memory, or storage than a laptop.
- **Access options:** OOD, Jupyter on HPC, terminal / SSH, remote IDEs, rg.arc.gwu.edu, etc.
- **Use HPC when:** the workflow is too slow or large for a laptop; you want repeatable execution; you want to batch or schedule runs; you want open models on shared infrastructure
- **Use AWS when:** run joint pipelines with researchers outside of GW, short term tasks the required specialized GPU architectures, ultra-high memory nodes, serverless triggers

Individual activity: Ollama on AWS

- Ollama is a way to run open models through a simple local API.
- In this workshop, we use Ollama to
 - run open models from Jupyter
 - avoid closed-model API keys
 - support repeated/scalable research workflows.

FYI – Typical setup for Jupyter on HPC:

- open terminal in Jupyter (top right)
- load modules (`ml ollama`)
- start the server with (`ollama serve &`)
- check installed models (`ollama list`);
- pull/install a model if needed (`ollama pull ...`)
- choose a model for the notebook workflow

Jupyter on AWS

1. <http://rg.arc.gwu.edu/>
2. Rocky Linux 8 Remote Desktop
3. `python3.12 -m venv my_new_env`
4. `source my_new_env/bin/activate`
5. `pip install notebook`
6. `jupyter notebook`

Sample pattern: calling Ollama from Jupyter

- load an input
- build a prompt from the template
- send the prompt to Ollama
- receive the response
- save the output and metadata

Code pattern: use Python to call the Ollama endpoint; start with one test example; then run over multiple rows or files

What is a minimum viable harness?

- **The smallest repeatable set up around an LLM task**
- **The harness is the code version of your context package**
 - input
 - prompt/template
 - script/notebook (code)
 - output file
 - log/metadata

Minimum viable harness example

Input: abstracts.csv

Prompt/template: Extract topic, population, method, main finding

Script/notebook: Jupyter notebook on HPC using Python + Ollama

Output file: lit_review_results.csv

Log/metadata: model, date, prompt version, input file, output file

Pod Activity: design your harness

Activity

Recommended time: 15 minutes

- Let's write pseudocode first
- input (or inputs, or batches of inputs)
- prompt/template
- script/notebook
- output file
- log/metadata
- optional: sketch a folder structure
- optional: note what should be checked by a human

Using the chatbot appropriately

Use the chatbot to: write snippets, explain code, debug errors, revise prompts, suggest ways to save outputs or metadata

Do not rely on the chatbot to: generate the whole workflow without understanding it, replace the scaffold, or make choices your pod has not already discussed

Pod activity: build the workflow

Activity

Recommended time: 30–40 minutes

1. Turn your harness plan into a working notebook workflow
2. Open new Jupyter Notebook (with Ollama still running in terminal if on HPC)
 - a. load input
 - b. format prompt(s)
 - c. call the model
 - d. save outputs
 - e. save metadata

Stretch: parameterize prompt/model, add validation, separate prompt text from code

Evaluate before you scale

- what counts as a good output?
- what failure modes matter?
- what needs human review?
- what should be logged?
- what data should not be sent to a model?

Activity: mini evaluation plan

Activity

Recommended time: 8–10 minutes

- one success criterion
- one failure criterion
- one human review rule
- one logging rule
- one privacy or ethics note

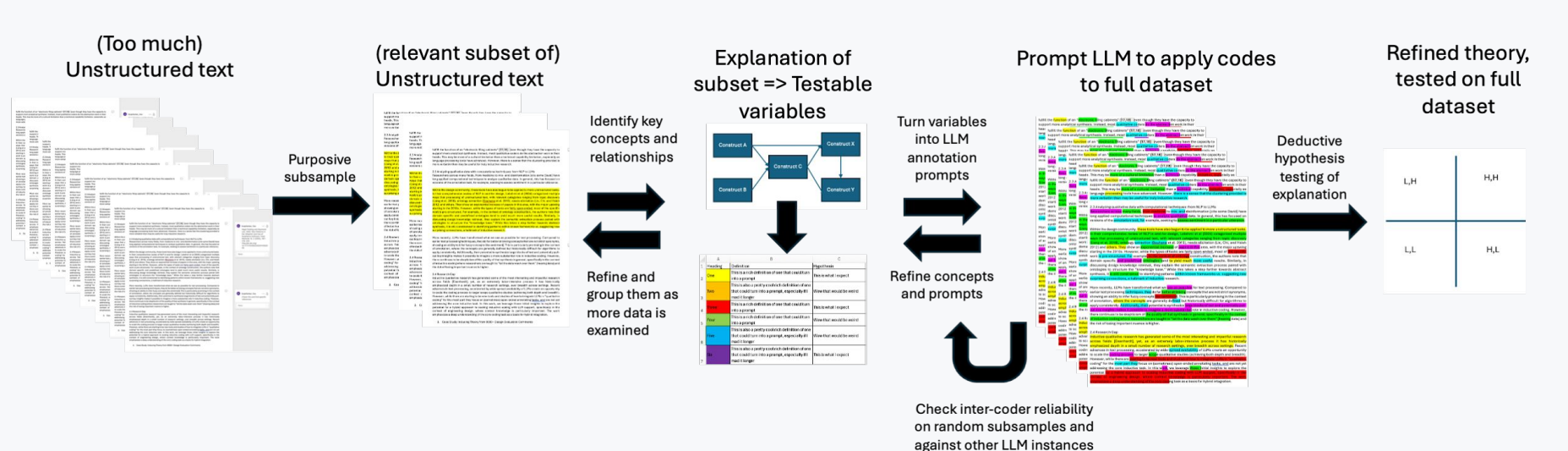
Pod Activity: more trustworthy?

Activity

Recommended time: 10 minutes

- How can a harness improve trustworthiness of research?
- How can a harness improve your practice of “open science” principles?
- What would you want to document about your harness to share with others (e.g., those who want to reproduce your research)?

Workflows, tasks, LLMs, Humans...



Task 1

Task 2

Task 3

Task 4

Task 5

Sample Options:

Human + LLM

LLM + Human + Human

Human + LLM + Human Eval

LLM + LLM Eval + Human Eval

HPC as workflow infrastructure

HPC is not just for “big models.”

Also use HPC for:

- larger datasets
- repeated runs
- scheduled or recurring jobs
- open/local models via Ollama
- reproducible execution
- saved logs and outputs

Consider **AWS** (also supported by the HPC team) when you need more flexibility, custom environments, cloud services, deployment, or external collaboration support

Preparing a workflow for HPC

- what script should run?
- where do inputs live?
- where should outputs go?
- what environment/modules are needed?
- what should be logged?
- what happens if something fails?

Jupyter vs. sbatch: when to use each

Use Jupyter when: explore interactively, test prompts on examples, inspect outputs step by step, debug quickly, develop the first version

Use sbatch when: run non-interactively, process many inputs, request resources, save logs, shard work, schedule later tasks, move to repeatable execution

What sbatch gives you

- runs in script files (not in Jupyter Notebooks)
 - run scripts non-interactively
 - request the right resources
 - capture stdout/stderr in log files
 - load modules and activate the correct environment
 - pass environment variables and tunable parameters
 - shard work with job arrays
 - schedule the next run or chain later tasks
 - support recurring or batch research tasks
-
- Handout has sbatch “cheat sheet”

What do we mean by “skills”?

In this workshop, a skill is a reusable part of a workflow.

- build a prompt from a row of data
- run an Ollama call
- parse structured output
- validate JSON
- flag uncertain cases
- save results
- write run metadata

Agents as workflows built from skills

Instead of thinking “autonomous AI scientist,” think:

- repeated tasks
- reusable skills
- logging and checks
- human review points
- orchestration across steps

A skill can be for example:

- an LLM-based function
- a deterministic function
- a validation/check function
- a save/logging function
- a routing/flagging step

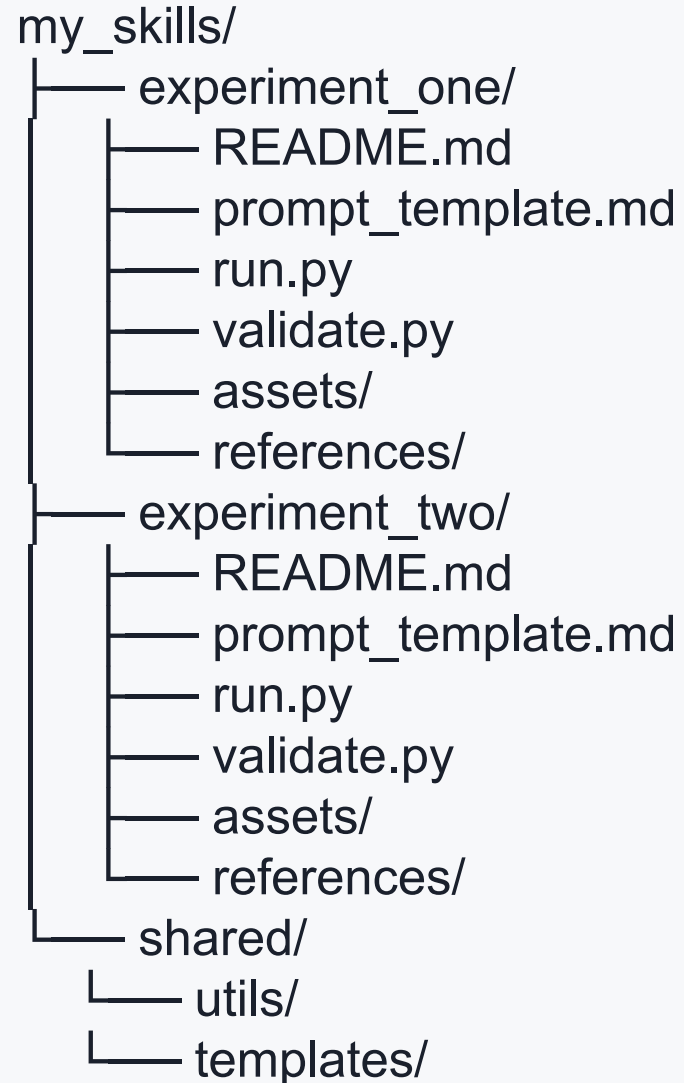
Skills.md

- In tools like Codex, Claude Code, or Kiro, a similar idea of skills that may be expressed through natural-language skill files or instructions, with more of the underlying implementation abstracted from the user.
- For reproducible and trustworthy research, we want skills to be **explicit**, **inspectable**, and **modifiable** rather than hidden behind abstraction, so we write them as code to be executed (which can include non-deterministic LLM functions like we have been writing today).

Skills example

```
def run_workflow(records):  
    results = []  
  
    for record in records:  
        prompt = build_prompt(record)  
        raw_output = run_ollama_call(prompt)  
        parsed_output = parse_structured_output(raw_output)  
        checked_output = validate_required_fields(parsed_output)  
  
        results.append({  
            "id": record["id"],  
            "output": checked_output  
        })  
  
    save_results(results)  
    write_metadata(results)
```

Sample skills structure



Skills structure

README.md

- Explains the skill to people:
 - what it does
 - when to use it
 - inputs/outputs
 - where it fits in the workflow

template.md

- Explains the task to the model:
 - what to do
 - how to format the output
 - what constraints to follow
 - parts of the minimum viable context package (from Day 1)

How to decompose your existing workflow into skills

Now return to the workflow your pod already built.

1. write your workflow as 5–8 plain-language steps
2. mark which steps require (or would benefit from) an LLM
3. mark which steps are deterministic or rule-based
4. identify where a human should review or approve
5. look for repeated steps that could become reusable skills
6. name each reusable part as a skill

Example: from workflow to skills

Workflow:

1. extract study information from article abstracts
2. load the abstracts file
3. build a prompt for one abstract
4. send the prompt to Ollama
5. parse the response into a structured format
6. validate required fields
7. flag incomplete outputs for human review
8. save results and metadata

Example Skills: load_inputs, build_prompt, run_ollama_call, parse_structured_output, validate_required_fields, flag_for_review, save_results, write_metadata

Pod Activity: human-in-the-loop design

Activity

Recommended time: 3 to 5 minutes

Discuss as a pod...

- what should be reviewed by a human?
- what should never be fully automated?
- when should outputs be flagged?
- what uncertainty should be documented?
- who is responsible for checking results?

What to do next

- improve your prompt/template
- add better structured outputs
- clean up your notebook into a script
- test on a larger sample
- add a validation check
- draft a real sbatch file
- refactor repeated steps into skills
- organize your skills for reuse across codebases

And keep going...

“Tests, commit histories, and repository structure are not separate from context engineering; they are context engineering at the timescale of a codebase.”

Lorena Barba, 2026